

ROADRESQ

Monitoring & Observability Plan

Version 1.0 • Production Health, Metrics, Logs, Traces & Alerting

Scope: Web • API • Mobile • Database • Redis • Workers • WebSockets • Payments • Maps • Notifications

Objective: Detect problems early, explain failures quickly and measure real customer outcomes

MEASURE → DETECT → CORRELATE → ALERT → MITIGATE → LEARN

Document Control

Field	Value
Document	Monitoring & Observability Plan
Product	RoadResQ
Version	1.0
Status	Production observability baseline
Audience	Engineering, DevOps, QA, Security, Support and Operations
Related documents	Deployment & DevOps Plan • Production Operations Runbook • System Architecture • Security Threat Model • Testing & QA Plan

1. Purpose

This document defines how RoadResQ observes production behavior across infrastructure, applications, databases, realtime systems, background processing and business workflows. The plan combines metrics, logs, traces, synthetic checks and business indicators so that operators can answer three questions quickly: Is the system healthy? What is failing? Is the customer experience being affected?

2. Observability Principles

- Observe both technical health and business outcomes.
- Use correlation IDs to connect requests, jobs, database activity and user journeys.
- Prefer actionable alerts over noisy alerts.
- Use structured logs instead of unsearchable text-only messages.
- Keep sensitive data out of telemetry.
- Measure latency using percentiles, not only averages.
- Make production releases observable by tagging telemetry with release/version identifiers.
- Retain enough history for troubleshooting and trend analysis while following approved privacy/retention rules.

3. Observability Model

RoadResQ uses five complementary telemetry types:

Signal	Answers	Examples
Metrics	How much/how often?	Request rate, error rate, latency, CPU, queue depth
Logs	What happened?	Booking transition, API error, payment webhook result
Traces	Where did time/failure occur?	API → matching → PostGIS → notification
Events	What state changed?	Booking accepted, provider arrived, payment confirmed
Synthetic checks	Does the critical journey work?	Create request → match → booking/status verification

4. Target Architecture

Application / Infrastructure → Telemetry SDK/Agents → Collector/Transport → Metrics Store + Log Store + Trace Store → Dashboards + Alert Engine → On-call / Incident Management

A managed observability platform may be used initially to reduce operational overhead. The design should remain vendor-neutral at the application instrumentation layer where practical.

5. Correlation & Context

- Generate/propagate a request ID for each inbound API request.
- Propagate correlation IDs into background jobs.
- Attach booking ID, provider ID or user ID only when operationally justified and access-controlled.
- Include release version, environment and service name in telemetry.
- Use trace/span IDs when distributed tracing is enabled.
- Never place authentication tokens, passwords, OTPs, payment secrets or raw sensitive documents into logs.

Recommended context fields:

```
timestamp | environment | service | version | request_id | trace_id | route | method | status_code | duration_ms |  
error_code | region/zone where appropriate
```

6. Service Inventory

Service	Key signals
Web	Availability, client errors, page/API latency, JS errors, web vitals
API	RPS, 4xx/5xx, p50/p95/p99 latency, saturation
Worker	Queue depth, job throughput, retries, failures, execution time
WebSocket	Connections, reconnects, event latency, dropped/error events
PostgreSQL/PostGIS	Connections, CPU, memory, storage, locks, slow queries, query latency
Redis	Memory, hit ratio where relevant, evictions, latency, connections, queue state
Object storage	Upload/download success, latency, capacity, access errors
Payments	Success/failure/pending, webhook latency, reconciliation mismatches
Maps	Request volume, errors, latency, quota/rate limits
OTP/Notifications	Delivery success/failure, provider latency, retries

7. Golden Signals

For every critical service, monitor the four golden signals: latency, traffic, errors and saturation.

Signal	RoadResQ examples
Latency	API p95/p99, matching duration, payment response, WebSocket event delay
Traffic	Requests/min, active bookings, location updates, WebSocket connections
Errors	HTTP 5xx, failed jobs, DB errors, payment webhook failures
Saturation	CPU, memory, DB connections, Redis memory, queue backlog

8. Application Metrics

- API request count by route/method/status.
- Request latency by route.
- Authentication success/failure and OTP verification outcomes.
- Booking request, matching, acceptance, cancellation and completion counts.
- Provider acceptance/rejection rates.
- Estimate approval/rejection and extra-work approval rates.

- Invoice creation and payment state transitions.
- Notification delivery and retry outcomes.
- Support ticket creation/resolution metrics.
- Unhandled exceptions and application error codes.

9. Business & Product Observability

Technical uptime alone is insufficient. RoadResQ must observe whether the core roadside assistance journey actually works.

Business metric	Purpose
Successful assistance jobs	North Star operational outcome
Request-to-match rate	Measures marketplace matching health
Time to provider acceptance	Measures dispatch responsiveness
ETA accuracy	Measures provider/location quality
Request cancellation rate	Detects friction or supply problems
Job completion rate	Measures end-to-end reliability
Payment success/reconciliation rate	Protects financial correctness
Repeat usage	Measures customer trust/value
Provider acceptance/completion	Measures provider ecosystem health
Average rating	Measures service quality

10. Customer Journey Monitoring

Track a request through the critical funnel:

REQUESTED → MATCHING → ACCEPTED → EN_ROUTE → ARRIVED → INSPECTION → ESTIMATE → APPROVAL → IN_PROGRESS → COMPLETED
→ INVOICE → PAYMENT → REVIEW

- Measure duration between major states.
- Detect abnormal increases in bookings stuck in a state.
- Alert on systemic state-machine failures.
- Use synthetic transactions to verify the journey without using real customer data.

11. Provider Journey Monitoring

PROVIDER ONLINE → REQUEST RECEIVED → ACCEPT/REJECT → EN_ROUTE → ARRIVED → JOB → COMPLETION → EARNINGS

- Monitor provider notification latency.
- Monitor request acceptance rate.
- Detect abnormal provider-location freshness.
- Measure provider completion and cancellation behavior.
- Identify geographic supply gaps through aggregate metrics.

12. Logging Standard

All production logs should be structured and machine-readable.

Example event shape:

```
{ "timestamp": "...", "level": "INFO", "service": "booking-api", "event": "booking.status_changed", "request_id": "...", "booking_id": "...", "from": "ACCEPTED", "to": "EN_ROUTE", "version": "..."} }
```

Recommended levels: DEBUG only when controlled; INFO for important lifecycle events; WARN for recoverable anomalies; ERROR for failed operations; FATAL/CRITICAL only for severe process-level conditions.

13. What Must Not Be Logged

- Passwords and password-reset secrets.
- OTP values.
- JWTs, session tokens or API secrets.
- Payment card data or sensitive payment credentials.
- Private provider identity documents.
- Unnecessary raw customer/provider location history.
- Full request bodies containing sensitive information.
- Authentication headers or signed URLs.

14. Error Taxonomy

Class	Examples	Response
Validation	Invalid vehicle/request data	Track rate; usually no alert
Authorization	Forbidden object access	Monitor spikes; security review if anomalous
Dependency	Maps/payment/notification unavailable	Alert if impact threshold exceeded
Database	Timeout, connection exhaustion, lock contention	Alert on sustained/critical impact
Business state	Invalid booking transition	High-priority application alert
Infrastructure	CPU/memory/disk/network issue	Infrastructure alert
Security	Suspicious access/traffic	Security alert + investigation

15. Distributed Tracing

Tracing should be introduced for critical cross-component journeys rather than every internal detail by default.

- API request → authentication → booking service.
- Emergency request → matching → geospatial query → dispatch.
- Booking event → worker → notification provider.
- Payment request → gateway → webhook → reconciliation.
- Upload request → authorization → object storage.

Use sampling to control cost, but retain 100% of traces for selected error/slow-request cases where the platform supports tail-based or targeted sampling.

16. Database Observability

- Connection count/utilization.
- Query latency and slow-query rate.
- Lock waits and long-running transactions.
- CPU, memory and storage.

- Index health and query-plan regressions.
- PostGIS query performance for provider matching.
- Replication/backup health where applicable.
- Migration execution and failure events.

17. Redis Observability

- Memory usage and fragmentation.
- Evictions.
- Command latency.
- Connections.
- Cache hit/miss where meaningful.
- Queue depth and processing latency.
- Lock/coordination anomalies.
- Redis availability/errors.

18. Worker & Queue Observability

Metric	Why it matters
Queue depth	Backlog and capacity
Oldest job age	Detects processing delay
Jobs/sec	Throughput
Success/failure rate	Reliability
Retry count	Dependency/application instability
Execution duration	Worker performance
Dead-letter count	Persistent failures needing review

19. WebSocket & Location Observability

- Active connections.
- Connection/reconnection rate.
- Authentication failures.
- Event publish and delivery latency.
- Dropped/error event count.
- Provider location update frequency.
- Stale-location percentage for active jobs.
- Server resource usage associated with realtime connections.

Critical principle: database/API state remains authoritative. Realtime telemetry should detect delivery problems without making the WebSocket layer the sole source of truth.

20. External Dependency Monitoring

Dependency	Monitor
Maps/geocoding/routing	Latency, error rate, quota, fallback behavior
Payment gateway	Success/error/pending, webhook delivery, reconciliation
OTP/SMS	Delivery, provider errors, latency, retry rate
Push notifications	Delivery/error metrics where available
Email	Delivery failures, bounce/provider errors
Object storage	Availability, upload/download errors, latency

21. Synthetic Monitoring

Synthetic checks should validate critical flows without creating real financial/customer impact.

- Health endpoint check.
- Read-only API authentication check.
- Test booking flow in a dedicated synthetic environment.
- Provider availability/matching test using test entities.
- Realtime connection and event delivery test.
- Notification test endpoint/provider sandbox where supported.
- Payment sandbox test for payment integrations.

22. Dashboards

Dashboard	Primary audience	Core panels
Executive/Product	Product/operations	Jobs, match rate, completion, cancellations, ratings
Platform	DevOps	Availability, latency, errors, saturation
API	Backend	Routes, errors, latency, dependency calls
Database	Backend/DevOps	Connections, slow queries, locks, storage
Realtime	Backend/DevOps	Connections, event latency, stale location
Workers	Backend/DevOps	Queue depth, retries, failures, throughput
Payments	Finance/Ops	Success, pending, webhook and reconciliation
Security	Security	Auth anomalies, suspicious traffic, privileged events

23. Alerting Philosophy

- An alert must have an owner and an expected action.
- Alert on sustained conditions, not every transient spike.
- Use severity levels aligned with the production incident process.
- Avoid duplicate alerts for the same root cause.
- Include dashboard/runbook references in alert descriptions.
- Automatically resolve alerts when the underlying condition clears where possible.

24. Initial Alert Catalog

Alert	Severity	Initial condition concept
API unavailable	SEV-1	Critical synthetic/availability failure
5xx spike	SEV-1/2	Sustained critical error rate above baseline
Latency degradation	SEV-2	p95/p99 materially above target
DB connection exhaustion	SEV-1/2	Pool/DB capacity near unsafe limit
DB storage critical	SEV-1/2	Safe capacity threshold breached
Redis unavailable	SEV-1/2	Critical dependency unavailable
Queue backlog critical	SEV-2	Backlog/oldest-job age exceeds target
Worker failure spike	SEV-2	Sustained failure/retry rate
Realtime degradation	SEV-2	Event/update failure or latency threshold
Payment reconciliation mismatch	SEV-1/2	Financial state mismatch detected
Backup failure	SEV-2	Required backup job failed
Security anomaly	SEV-1/2	Confirmed/high-confidence suspicious activity

25. Alert Payload Standard

Every actionable alert should contain:

- Alert name and severity.
- Affected service/environment.
- Current value and expected baseline/threshold.
- Start time and duration.
- Likely impact.
- Dashboard link.
- Relevant runbook section.
- Suggested first diagnostic actions.

26. SLO / SLI Framework

Service	Example SLI	SLO direction
API availability	Successful valid requests	High availability target
API latency	p95/p99 response time	Within approved latency target
Booking reliability	Valid requests reaching stable assignment/completion	High success rate
Realtime	Active jobs receiving timely updates	High delivery/freshness target
Payments	Correctly reconciled payment outcomes	Very high correctness target
Notifications	Transactional notification delivery	High successful delivery target

Numeric SLOs should be finalized from production baselines, business criticality and load-testing results rather than copied blindly from generic benchmarks.

27. Release Observability

- Tag telemetry with release version.

- Compare error/latency before and after deployment.
- Annotate dashboards with deployment timestamps.
- Monitor database migration duration/errors.
- Use canary/rolling comparison when supported.
- Automatically trigger smoke/synthetic checks after release.

28. Capacity & Performance Monitoring

- Track CPU, memory, network and disk trends.
- Track database growth and query performance.
- Track Redis memory trends.
- Track queue growth versus worker throughput.
- Track active bookings and realtime connection growth.
- Monitor third-party API quotas and costs.
- Review capacity before major campaigns/events or geographic expansion.

29. Cost Observability

- Cloud compute cost by environment/service.
- Database and storage cost.
- Logs/metrics/traces ingestion cost.
- Maps/routing API usage and quota.
- SMS/OTP and notification cost.
- Payment provider fees where operationally relevant.

Telemetry should be useful without becoming an uncontrolled cost center. Apply retention, aggregation and sampling intentionally.

30. Privacy & Data Governance

- Define telemetry retention by data class.
- Minimize personally identifiable information in logs.
- Restrict access to production telemetry.
- Apply encryption in transit and at rest.
- Support deletion/redaction procedures where legally and technically required.
- Audit privileged access to sensitive operational data.
- Do not use production customer data in development dashboards.

31. Incident Investigation Workflow

ALERT → IDENTIFY SERVICE → CHECK DASHBOARD → CORRELATE REQUEST/TRACE → CHECK RECENT CHANGE → CHECK DEPENDENCIES → ASSESS BLAST RADIUS → MITIGATE → VERIFY → DOCUMENT

- Start with symptoms, then trace toward the root cause.
- Compare current telemetry with a known-good baseline.
- Check deployment markers and infrastructure changes.
- Use booking IDs/request IDs for controlled transaction-level investigation.

- Avoid querying or exposing unnecessary customer data.

32. Observability for Common Failures

Failure	Primary signals
API outage	Synthetic checks, 5xx, load balancer, process health
Slow API	p95/p99, traces, DB query latency, dependency latency
Matching failure	Booking funnel, matching duration, PostGIS, provider availability
Booking stuck	State transition events, worker queue, DB locks, realtime
Payment issue	Gateway responses, webhook latency, reconciliation metrics
Notification delay	Queue age, worker retries, provider delivery
Tracking stale	WebSocket health, location freshness, mobile connectivity indicators
Database overload	Connections, CPU/IO, slow queries, locks

33. Telemetry Retention Baseline

Retention should be configurable by telemetry type and approved privacy requirements. A practical baseline is to keep high-volume debug data short, operational metrics long enough for trend analysis, and incident/security evidence according to policy and legal requirements.

Data	Retention approach
High-volume application logs	Short operational window + aggregated metrics
Error logs	Longer than general logs for troubleshooting
Metrics	Sufficient history for SLO/capacity trends
Traces	Sampled retention; longer for important incidents if policy permits
Audit/security logs	Policy-defined extended retention
Location telemetry	Minimum required for operations; strict retention control

34. On-Call Operations

- Maintain an on-call owner for production-critical alerts.
- Every SEV-1/SEV-2 alert should map to a runbook.
- Escalate when the primary responder cannot mitigate within the agreed window.
- Do not acknowledge an alert without either investigating or assigning it.
- Review recurring alerts for automation or engineering fixes.

35. Monitoring Maturity Roadmap

Stage	Capabilities
MVP	Health checks, structured logs, core metrics, error tracking, basic alerts
Production v1	SLOs, dashboards, queue/DB/realtime/payment monitoring, synthetic checks
Scale	Distributed tracing, advanced anomaly detection, autoscaling signals, capacity forecasting
Advanced	Business journey intelligence, automated remediation, predictive capacity/risk analysis

36. Monitoring Implementation Checklist

Area	Ready?
Structured application logging	■
Request/correlation IDs	■
API golden signals	■
Database monitoring	■
Redis monitoring	■
Worker/queue monitoring	■
WebSocket/realtime monitoring	■
Payment/reconciliation monitoring	■
External dependency monitoring	■
Synthetic critical journey checks	■
Production dashboards	■
Actionable alerts	■
SLO/SLI baseline	■
Telemetry privacy/retention controls	■
Incident/runbook integration	■

37. Definition of Done — Observability

- Critical services emit useful metrics and structured logs.
- Critical customer/provider journeys have business-level telemetry.
- Request/job/trace correlation is available for investigation.
- Dashboards show health, performance and business outcomes.
- Alerts are actionable, owned and connected to runbooks.
- Database, Redis, workers, realtime and integrations are observable.
- Payment correctness and reconciliation are monitored.
- Sensitive information is excluded or controlled in telemetry.
- SLOs and retention are documented.
- Production incidents can be investigated without guessing.

38. Final Observability Standard

RoadResQ observability must provide a continuous operational picture from infrastructure to customer outcome. A healthy system is not merely one with low CPU or no crashed processes; it is one where customers can request assistance, providers can receive and accept jobs, live status remains trustworthy, work completes correctly and payments reconcile safely. Monitoring should make that state measurable, explainable and actionable.